

Service Overview

ภาพรวมของบริการ (Service Overview)

EmergencyResourceRequest Service

1. Service Owner

นายชนกันต์ ทองรอง รหัสนักศึกษา 6609611816 ภาคปกติ

2. Service Purpose

EmergencyResourceRequest Service เป็นบริการที่รับผิดชอบการรับเรื่อง รวบรวม และจัดการคำร้องขอทรัพยากรความช่วยเหลือ เช่น อาหาร น้ำ ยา จากผู้ประสบภัยและหน่วยงานส่วนหน้า โดยทำหน้าที่แปลงข้อมูลคำร้องขอให้อยู่ในรูปแบบที่มีโครงสร้าง (Structured Data) เพื่อส่งต่อให้ทีมกู้ภัยนำไปใช้วางแผนการกระจายสิ่งของได้อย่างมีประสิทธิภาพ

3. Pain Point ที่แก้ไข

ในสถานการณ์ภัยพิบัติ ข้อมูลการขอความช่วยเหลือมักกระจายไม่ระบุพิกัดที่ชัดเจน และมีการแจ้งซ้ำซ้อนเนื่องจากอินเทอร์เน็ตไม่เสถียร ทำให้หน่วยกู้ภัยไม่ทราบว่าพื้นที่ใดมีความต้องการเร่งด่วน บริการนี้จึงเข้ามาช่วยจัดระเบียบข้อมูล แยกแยะระดับความสำคัญ และป้องกันการบันทึกข้อมูลซ้ำซ้อน เพื่อให้ความช่วยเหลือไปถึงผู้ที่ต้องการจริงๆ ได้อย่างรวดเร็ว

3. Target Users

1. Citizen (ประชาชนผู้ประสบภัย): ผ่านทางแอปพลิเคชันหรือเว็บ Frontend
2. Rescue Team: เจ้าหน้าที่ลงพื้นที่หรือทีมกู้ภัยที่ต้องการดูรายการคำขอและกดรับงาน

4. Service Boundary

- In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ):
 - รับและจัดเก็บข้อมูลคำร้องขอความช่วยเหลือและทรัพยากร (Resource Requests)
 - จัดการสถานะการดำเนินการของคำร้องขอ
 - ตรวจสอบและคัดกรองข้อมูลคำร้องขอซ้ำซ้อน (Duplicate Detection / Idempotency)
 - จัดเก็บพิกัด (Location) และระดับความเร่งด่วน (Priority) ของแต่ละคำขอ
- Out-of-scope / Not Responsible For (ไม่รับผิดชอบ):
 - การจัดการคลังสินค้าหรือสต็อกสิ่งของบริจาค (Inventory Management)
 - การจัดการข้อมูลทีมกู้ภัยหรือการนำทางรถบรรทุก (Fleet Routing)
 - การจัดการข้อมูลภาพรวมของเหตุการณ์ภัยพิบัติ (Incident Master Data)

5. Autonomy / Decision Logic

บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ:

- การตรวจจบข้อมูลซ้ำซ้อน (Idempotency): หากมีคำร้องขอที่มีข้อมูลผู้แจ้งและพิกัดเดิมส่งเข้ามาในเวลาใกล้เคียงกัน บริการจะปฏิเสธหรือรวมเป็นคำขอเดียวโดยอัตโนมัติ
- การอัปเดตสถานะ: เมื่อมีการระบุตัวทีมกู้ภัย (`delivery_team_id`) เข้ามาในระบบ บริการสามารถปรับสถานะจาก **NEW** เป็น **IN_PROGRESS** ได้ทันที

การตัดสินใจอิงจาก:

- ความถูกต้องของ incident id จาก incident tracking service
- ความสมบูรณ์ของรายการสิ่งของที่ขอ (Items payload)

บริการสามารถตัดสินใจได้เองภายใต้ business rules ที่กำหนด โดยไม่ต้องรอ/ต้องรอการอนุมัติจากมนุษย์ในกรณีปกติ

6. Owned Data

- **Resource Request Master Data (ข้อมูลคำร้องขอ):** ข้อมูลรายละเอียดคำขอ เช่น **items, requester** (ผู้แจ้ง), **location, priority** เป็นข้อมูลแกนหลักที่เกิดจาก Action ภายในบริการนี้ จึงต้องเป็นเจ้าของเพื่อให้แก้ไขและดึงข้อมูลได้อย่างรวดเร็ว
- **Request Operational State (สถานะการดำเนินการ):** ข้อมูล **status (NEW, IN_PROGRESS, CLOSED)** และ **time_requested** เป็น state ที่เชื่อมโยงกับ Lifecycle ของคำร้องขอ บริการนี้จึงต้องรับผิดชอบเพื่อป้องกันไม่ให้ Service อื่นมาแก้ไขสถานะโดยพลการ
- **Idempotency Keys / Request Logs (ข้อมูลป้องกันการซ้ำซ้อน):** ข้อมูลสำหรับตรวจสอบประวัติการส่ง Request ซ้ำซ้อน ต้องอยู่ภายใต้บริการนี้เพื่อรับประกันความถูกต้องของการสร้างข้อมูลใหม่

7. Linked Data (Reference Only)

- **incident_id:** อ้างอิงเหตุการณ์ภัยพิบัติจาก Incident Service เพื่อระบุว่าคำขอนี้เกิดในภัยพิบัติรอบไหน
- **delivery_team_id:** อ้างอิงทีมกู้ภัยจาก Rescue Team Service (ระบบจะไม่เก็บชื่อหรือเบอร์โทรทีมกู้ภัยไว้ที่นี่ แต่จะเก็บแค่ ID)
- **item_id:** อ้างอิงถึงสินค้าที่มีอยู่ใน stock (จะอยู่ใน field jsonb items) โดยจะอิงไปถึง service ที่เกี่ยวข้องกับการจัดการ stock

8. Non-Functional Requirements

- **Idempotency:** API สำหรับสร้างคำขอ (**POST**) ต้องรองรับ Idempotency Key ป้องกันการสร้างข้อมูลซ้ำเมื่อผู้ประสบกดส่งซ้ำเพราะเน็ตหลุด
- **Availability:** เนื่องจากเป็นระบบที่ใช้งานในภาวะฉุกเฉิน API ฝั่งรับเรื่อง (Create Request) ต้องสามารถทำงานได้ (High Availability) แม้ Service อื่นที่เกี่ยวข้องจะล่ม โดยต้องมี fallback value เช่น STATUS เป็น UNDECIDED
- **Security:** API Endpoint สำหรับดึงข้อมูลไปแสดงผลหรืออัปเดตสถานะ (ที่ใช้โดยทีมกู้ภัย) ต้องมีการยืนยันตัวตน (Authentication) เพื่อป้องกันข้อมูลส่วนบุคคลของผู้ประสบภัยรั่วไหล

Sync Contract

Synchronous Function Contract

Base URL:

https://re-fed69572302e455c92292715abb32750.ecs.ap-southeast-7.on.aws/

API Contract #1: Create Resource Request

ข้อมูลทั่วไป

- **Name:** Create a new resource request
- **Method:** POST
- **Path:** **/v1/resource**
- **Type:** Synchronous

คำอธิบาย

ใช้สำหรับสร้างคำร้องขอทรัพยากรและความช่วยเหลือใหม่จากผู้ประสบภัย โดยระบุรายการสิ่งของที่ต้องการและข้อมูลตำแหน่งที่ตั้ง

Request

Headers

- Content-Type: application/json
- Idempotency-Key: <uuid> (required)

Body:

```
{
  "description": "ต้องการความช่วยเหลือด่วน น้ำท่วมสูง",
  "requestFor": "CITIZEN",
  "items": [
    { "id": "1", "amount": 1 }
  ],
  "extraItems": [
    { "name": "ยาหม่อง", "amount": 3 }
  ],
  "from": {
    "name": "Somchai Maipong",
    "location": {
      "address": "32/1 Phichit",
      "description": "Behind 7-11",
      "latitude": 16.4425,
      "longitude": 100.3488
    }
  },
}
```

```
"contact": {  
  "phone": "012345678"  
}  
}  
}
```

Response

Success: 201

```
{ "id": "req-1234-uuid", "status": "ACCEPT" }
```

Error: 400, 409

```
{ "message": "Error description here" }
```

Dependency / Reliability

- ไม่เรียก service อื่นแบบ Synchronous
- ต้องทำ Idempotency check จาก Header
- Timeout: 10s

API Contract #2: List Requests

ข้อมูลทั่วไป

- **Name:** List requests by incident_id, status, and priority
- **Method:** GET
- **Path:** `/v1/resource/list-request`
- **Type:** Synchronous

คำอธิบาย

ใช้ดึงข้อมูลรายการคำร้องขอ โดยสามารถกรองตามรหัสเหตุการณ์ (incident_id), สถานะ (status), และระดับความสำคัญ (priority) ได้

Request

Query Params

- `incident_id` (uuid, required)
- `status` (string, optional)
- `priority` (string, optional)

Validation

- `status` $\in$ {OPEN, FULL, CLOSED}

- incidentId required

Response

Success: 200

```
[
  {
    "id": "req-1234-uuid",
    "items": [{ "id": "1", "amount": 1 }],
    "extra_items": [{ "name": "ยาหม่อง", "amount": 3 }],
    "from": {
      "name": "Somchai Maipong",
      "location": {
        "address": "32/1 Phichit",
        "description": "Behind 7-11",
        "latitude": 16.4425,
        "longitude": 100.3488
      },
      "contact": { "phone": "012345678" }
    }
  }
]
```

Error: 404

```
{ "message": "Not found" }
```

Dependency / Reliability

- ไม่เรียก service อื่น
- Timeout: 30s

API Contract #3: Get Resource Request

ข้อมูลทั่วไป

- Name: Get resource request
- Method: GET
- Path: **/v1/resource/{request_id}**
- Type: Synchronous

คำอธิบาย

ใช้ดึงข้อมูลรายละเอียดเชิงลึกของคำร้องขอทรัพยากรเฉพาะรายการ (ระบุด้วย request_id)

Request

Path Params

- request_id (uuid, Required)

Validation

- request_id required

Response

Success: 200

```
{
  "id": "req-1234-uuid",
  "status": "NEW",
  "priority": "CRITICAL",
  "items": [{ "id": "1", "amount": 1 }],
  "extra_items": [{ "name": "ยาหม่อง", "amount": 3 }],
  "from": {
    "name": "Somchai",
    "location": { /* ... details ... */ },
    "contact": { "phone": "012345678" }
  }
}
```

Error: 404

```
{ "message": "Request not found" }
```

Dependency / Reliability

- ไม่เรียก service อื่น
- Timeout: 30s

API Contract #4: Assign Team to Request

ข้อมูลทั่วไป

- Name: Assign team to request
- Method: POST

- Path: `/v1/resource/{request_id}/assign`
- Type: Synchronous

คำอธิบาย

- ใช้สำหรับมอบหมายทีมกู้ภัย (Response Team) ให้จัดการคำร้องขอนี้ ระบบจะเปลี่ยนสถานะเป็น **IN_PROGRESS**

Request

Path Params

- **request_id** (uuid, Required)

Body

- `{ "team_id": "team-789-uuid" }`

Validation

- request_id required

Response

Success: 201

```
{
  "request_id": "req-1234-uuid",
  "team_id": "team-789-uuid",
  "status": "IN_PROGRESS",
  "assigned_at": "2026-03-22T12:00:00Z"
}
```

Error: 404

```
{ "message": "Error description" }
```

Dependency / Reliability

- Timeout: 30s

API Contract #5: Unassign Team

ข้อมูลทั่วไป

- Name: Unassign team

- Method: **PATCH**
- Path: **/v1/resource/{request_id}/unassign**
- Type: Synchronous

คำอธิบาย

- ใช้สำหรับยกเลิกการมอบหมายที่มูกู้ยออกจากคำร้องขอ และรีเซ็ตสถานะกลับเป็น **NEW**

Request

Path Params

- **request_id** (uuid, Required)

Validation

- request_id required

Response

Success: 200

```
{  
  
  "request_id": "req-1234-uuid",  
  "status": "NEW"  
}
```

Dependency / Reliability

- Timeout: 30s

API Contract #6: Close Resource Request

ข้อมูลทั่วไป

- Name: Close resource request
- Method: **PATCH**
- Path: **/v1/resource/{request_id}/close**
- Type: Synchronous

คำอธิบาย

- ใช้สำหรับอัปเดตสถานะของคำร้องขอเป็น **CLOSED** หลังจากที่มีมูกู้ยได้ดำเนินการช่วยเหลือ/จัดส่งทรัพยากรเสร็จสิ้นแล้ว

Request

- Path Params: **request_id** (uuid, Required)

Response

- Success (200):

```
{  
  
  "request_id": "req-1234-uuid",  
  
  "status": "CLOSED"  
}
```

Async Contract

Asynchronous Function Contract

Message Contract #1: Request Resource

ข้อมูลทั่วไป

- Message Name: ResourceRequestCreated
- Interaction Style: Request-Async Response (via Message Broker)
- Producer: EmergencyResourceRequest Service
- Consumer: Other services
- Channel/Queue: resource-request-event
- Version: v1

คำอธิบาย

เมื่อมีผู้ประสบภัยส่งคำร้องขอทรัพยากรเข้ามาใหม่และระบบทำการบันทึกลง Database สำเร็จ บริการนี้จะทำการส่ง Event ออกไปทันที เพื่อแจ้งให้ระบบอื่นรับทราบและนำข้อมูลไปดำเนินการต่อในส่วนของตนเอง (เช่น เช็กสต็อก หรือ จัดทีมกู้ภัย) โดยไม่ต้องรอให้ Service อื่นทำงานเสร็จ เพื่อรักษา High Availability ของระบบรับแจ้งเหตุ

Request

Message Headers

- traceId: string/uuid (optional - สำหรับใช้ track log ข้าม service)
- messageId: string
- version: string
- type: string

Message Body

```
{  
  "id": "req-1234-uuid",  
  "traceId": "trace-uuid-1111",  
  "incidentId": "8b9e4b0d-3b7e-4a0d-9d9f-3f5c6a2a3e10",  
  "priority": "UNDECIDED",  
  "requestFor": "CITIZEN",  
  "requesterName": "สมชาย ใจดี",  
  "phone": "0812345678",  
  "description": "ต้องการความช่วยเหลือด่วน น้ำท่วมถึงชั้น 2",  
  "address": "123 ม.4 ต.ในเมือง อ.เมือง จ.พิจิตร",  
}
```

```

"latitude": 16.4425,
"longitude": 100.3488,
"items": {
  "create": [
    { "itemId": "1", "amount": 1 }
  ]
},
"extraltems": {
  "create": [
    { "name": "ยาหม่อง", "amount": 3 }
  ]
},
"verified": true
}

```

Field Definition:

Field	Type	Required	Description
id	UUID	Y	รหัสคำร้องขอที่สร้างสำเร็จ (PK จากบริการของเรา)
traceld	string	Y	traceld ที่มาจาก server
incidentId	UUID	Y	รหัสอ้างอิงเหตุการณ์ภัยพิบัติ
requestFor	string	Y	ประเภทของคำร้องขอว่าสำหรับใคร
requesterName	string	Y	ชื่อผู้ขอ

phone	string	Y	เบอร์มือถือติดต่อ
description	string	N	ข้อมูลเพิ่มเติม
items	array<json>	N	รายการสิ่งของที่มีในระบบและจำนวนที่ต้องการ
extra_items	array<json>	N	รายการสิ่งของนอกระบบ (พิมพ์ชื่อเอง) และจำนวนที่ต้องการ
address	string	Y	ตำแหน่ง
longitude	number	Y	พิกัดจุดเกิดเหตุ (Latitude, Longitude)
latitude	number	Y	พิกัดจุดเกิดเหตุ (Latitude, Longitude)
verified	boolean	Y	incident id นี้มีอยู่จริงหรือไม่

Validation Rules

- request_id และ incident_id ต้องไม่เป็นค่าว่าง
- fields ที่ required ต้องมีค่า

Response

Message Headers

- Content-Type: application/json

Success Message Body

“DONE”

Message Contract #2: Request Resource to Prioritization

ข้อมูลทั่วไป

- Message Name: ResourceRequestCreated
- Interaction Style: Message broker
- Producer: EmergencyResourceRequest Service
- Consumer: Rescue Request Prioritize Service
- Channel/Queue: to-prioritize-service
- Version: v1

คำอธิบาย

เมื่อมีคำขอถูกเพิ่มเข้าระบบ Database จะส่ง body ไปยัง AWS SNS เพื่อให้ Rescue Request Prioritize Service นำคำขอไปจัดการ priority ของ request

Request

Message Body

```
{

  "requestId": "req-1234-uuid",

  "incidentId": "8b9e4b0d-3b7e-4a0d-9d9f-3f5c6a2a3e10",

  "requestType": "CITIZEN",

  "description": "ต้องการความช่วยเหลือด่วน น้ำท่วมถึงชั้น 2",

  "peopleCount": 1,

  "specialNeeds": [],

  "items": [

    { "name": "ยาหม่อง", "amount": 3 }

  ],

  "location": {

    "latitude": 16.4425,

    "longitude": 100.3488,

    "province": null,

    "district": null,

    "subdistrict": null,

    "addressLine": "123 ม.4 ต.ในเมือง อ.เมือง จ.พิจิตร"

  },

  "submittedAt": "2026-05-18T00:30:00.000Z"

}
```

Response

None

Service Data

Service Data

1) Resource Request Master Data (Owned by this service)

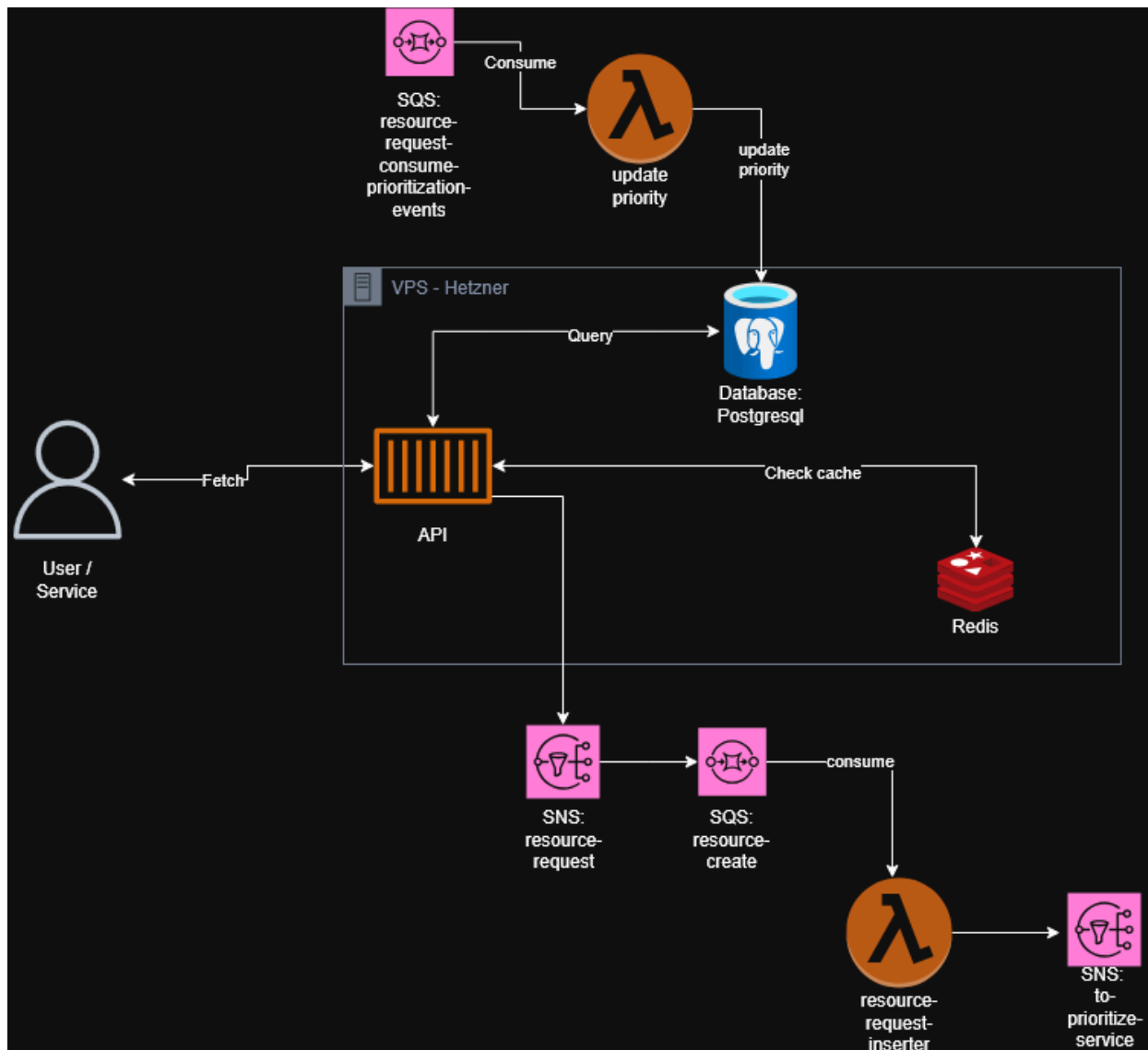
Field Name	Type	Required	Description	Example
id	UUID (PK)	Y	รหัสอ้างอิงของคำร้องขอ (Primary Key)	req-1234-uuid
incident_id	UUID	Y	รหัสอ้างอิงเหตุการณ์ภัยพิบัติ (เชื่อมกับ Incident Service)	8b9e4b0d-3b7e...
request_for	String	Y	ประเภทของการร้องขอสำหรับใคร	CITIZEN
priority	String (Enum)	Y	ระดับความเร่งด่วน (CRITICAL, NORMAL, LOW)	CRITICAL
requester	JSONB	Y	ข้อมูลผู้แจ้ง (ชื่อ, เบอร์ติดต่อฉุกเฉิน)	{"name": "Somchai", "phone": "012345678"}
location	JSONB	Y	พิกัดจุดเกิดเหตุ และคำอธิบายที่อยู่	{"latitude": 16.4425, "longitude": 100.3488}
status	String (Enum)	Y	สถานะปัจจุบัน (NEW, IN_PROGRESS, CLOSED)	NEW

delivery_team_id	UUID	N	รหัสทีมกู้ภัยที่รับหน้าที่จัดการ (จะอัปเดตภายหลัง)	team-789-uuid
requested_at	Timestamp	Y	วัน-เวลาที่มีการสร้างคำร้องขอนี้ขึ้นมา	2026-01-30T14:25:00Z

2) Request Line Item (Owned by this service)

Field Name	Type	Required	Description	Example
id	UUID (PK)	Y	รหัสอ้างอิงของรายการย่อย (Primary Key)	item-8899-uuid
request_id	UUID (FK)	Y	รหัสคำร้องขอหลัก	req-1234-uuid
item_type	String	Y	ประเภทสิ่งของ (STANDARD = มีในระบบ, EXTRA = นอกกระบบ)	EXTRA
item_ref_id	String	N	รหัสสิ่งของอ้างอิง (กรณีเป็น STANDARD)	1 (ปล่อยว่างถ้าเป็น EXTRA)
item_name	String	N	ชื่อสิ่งของ (กรณีเป็น EXTRA ที่ผู้ใช้พิมพ์เอง)	"ยาหม่อง"
amount_requested	Integer	Y	จำนวนที่ร้องขอ	3

Service Architecture



Service Architecture

Components

- **User (Client):** ผู้ใช้งานระบบที่ส่งคำขอ เพื่อดึงข้อมูลหรือสร้างคำร้องขอใหม่
- **REST API Server (Hetzner VPS via Coolify):** บริการ Elysia/Bun ที่ deploy บน Hetzner VPS ผ่าน Coolify ทำหน้าที่รับคำขอจาก User หรือ Service อื่น ๆ จัดการ Business logic, ตรวจสอบ idempotency key ผ่าน Redis, Query database และส่งต่อ Event คำร้องขอใหม่ไปยัง SNS
- **Redis (Hetzner VPS):** ระบบหน่วยความจำแคช (In-memory Cache) ที่ self-host บน Hetzner VPS เดียวกันกับ API ผ่าน Coolify ใช้สำหรับบันทึกและดึงข้อมูล idempotency key (Get / Set) เพื่อป้องกันการสร้างคำร้องขอซ้ำ และลดปริมาณการคิวรีฐานข้อมูลโดยตรง

- **PostgreSQL (Hetzner VPS):** ฐานข้อมูลเชิงสัมพันธ์ (Relational Database) ที่ self-host บน Hetzner VPS ผ่าน Coolify ทำหน้าที่จัดเก็บข้อมูลทรัพยากรและคำร้องขอต่างๆ รองรับการดึงข้อมูล (Query / Return data) จาก API Server และการเพิ่มข้อมูลใหม่ (Insert new request) จาก AWS Lambda ที่ connect กลับมาจาก AWS
- **SNS: resource request topic:** บริการกระจายข้อความแบบ Pub/Sub ทำหน้าที่รับ Event "Request Resource" จากบริการหลัก (API Server บน Hetzner VPS) และนำไป Publish กระจายต่อให้ระบบที่เกี่ยวข้อง
- **SQS: resource request:** คิวสำหรับพักข้อความ (Message Queue) ที่รับ Event มาจาก SNS เพื่อจัดลำดับและเก็บข้อความไว้รอการประมวลผล ป้องกันไม่ให้ระบบหลังบ้านทำงานหนักเกินไปเมื่อมีคำขอเข้ามาพร้อมกันจำนวนมาก
- **AWS Lambda (Async Worker):** ฟังก์ชันที่ทำงานแบบ Serverless คอยทำงานอยู่เบื้องหลังทำหน้าที่ดึงข้อความคำร้องขอ (Consume) จาก SQS มาประมวลผล และทำการบันทึกข้อมูลคำร้องขอใหม่ (Insert new request) ลงในฐานข้อมูล PostgreSQL บน Hetzner VPS โดย connect กลับมาจาก AWS
- **Hetzner VPS (Coolify):** เซิร์ฟเวอร์ที่ self-host บน Hetzner ซึ่ง deploy และจัดการผ่าน Coolify ครอบคลุมบริการหลัก ได้แก่ REST API Server (Elysia/Bun), Redis และ PostgreSQL เพื่อจำกัดขอบเขตการเข้าถึงและลดค่าใช้จ่ายเทียบกับ managed cloud services

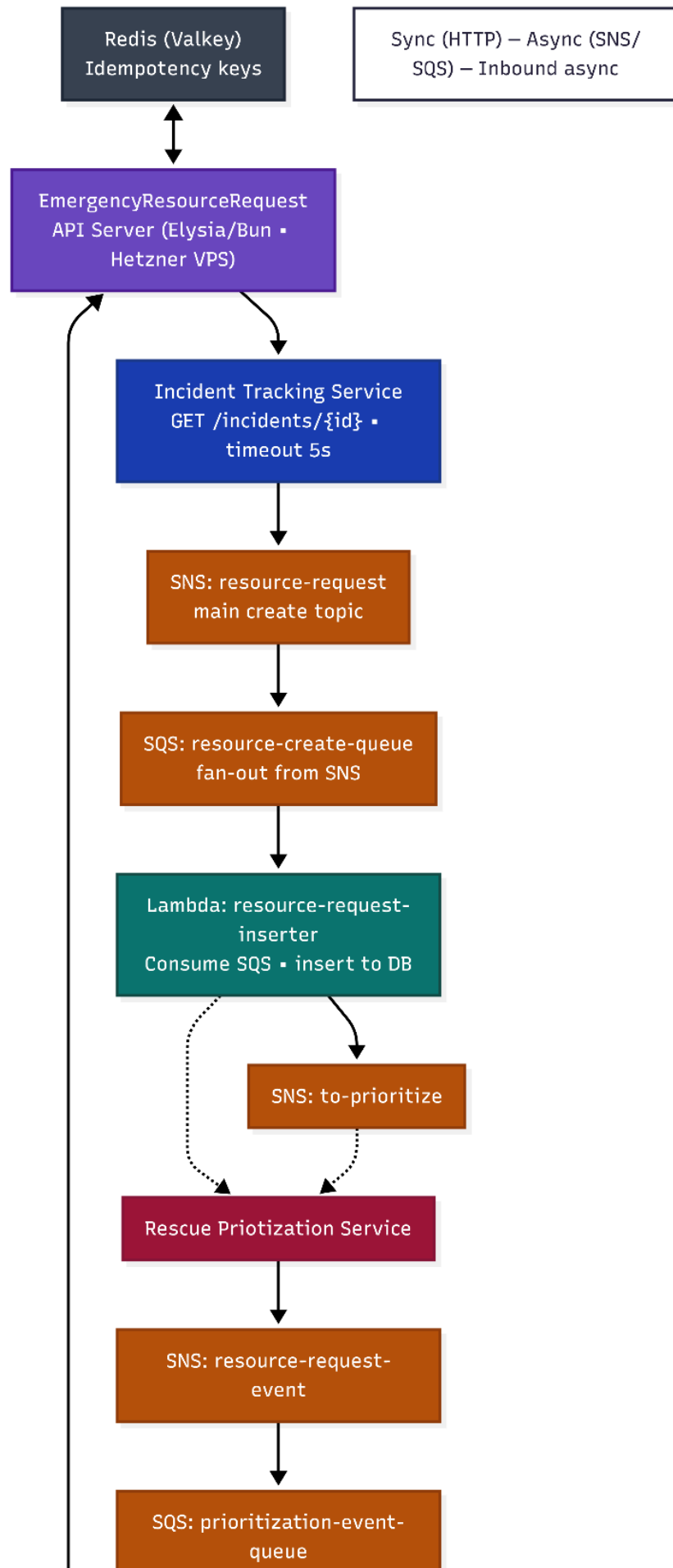
Explanation

สถาปัตยกรรมของ **Resource Request Service** นี้ ถูกออกแบบมาเพื่อเพิ่มประสิทธิภาพในการรองรับโหลดข้อมูลระดับหนึ่ง มีการนำระบบ Cache มาช่วยลดภาระของฐานข้อมูลหลัก การทำงานแบ่งออกเป็น 2 ส่วนหลัก ดังนี้:

- **Synchronous Flow (สำหรับการเรียกดูและค้นหาข้อมูล):** เมื่อ User ส่งคำสั่ง "Fetch" เข้ามายัง API Server บน Hetzner VPS ระบบจะทำงานแบบตอบกลับทันที ตรวจสอบ idempotency key ใน Redis ก่อน แล้วจึงติดต่อกับ PostgreSQL โดยตรง นำผลลัพธ์ตอบกลับไปยัง User
- **Asynchronous Flow (สำหรับการสร้างคำร้องขอทรัพยากรใหม่):** เมื่อระบบได้รับคำสั่งสร้างคำร้องขอใหม่ (Request Resource) แทนที่ API Server จะเขียนข้อมูลลงฐานข้อมูลโดยตรง (ซึ่งอาจทำให้ระบบเกิดการล็อกหรือล่าช้า) API Server จะส่งข้อมูลนั้นเป็น Event ไปยัง SNS (resource request topic) แทน จากนั้น SNS จะทำการกระจายข้อความ (Publish) ไปพักไว้ที่คิว Amazon SQS เมื่อข้อความอยู่ในคิวแล้ว AWS Lambda จะคอยดึงข้อความเหล่านั้น (Consume) ไปประมวลผลตามลำดับแบบเบื้องหลัง แล้วจึงนำข้อมูลที่ประมวลผลเสร็จแล้วไปบันทึกลงฐานข้อมูล PostgreSQL บน Hetzner VPS (Insert new request)

Service Interaction

Service Interaction



Service / Component	ประเภท Interaction	เกิดเมื่อ	ทิศทาง	หมายเหตุ
Incident Tracking Service	Synchronous (HTTP GET)	ทุกครั้งที่เราสร้างคำร้องขอใหม่ (POST /v1/resource)	Outbound	timeout 5s, fail-safe: verified=false
Redis (Valkey)	Synchronous (In-memory)	ทุก POST ที่มี Idempotency-Key header	Outbound	NX SET expire 30s
PostgreSQL	Synchronous (SQL) — อ่าน / Async — เขียน	อ่าน: GET/PATCH endpoints / เขียน: ผ่าน Lambda	Internal	Hetzner VPS
SNS: resource-request	Asynchronous (Publish)	หลัง API รับ POST / สร้างคำขอใหม่	Outbound	main create topic
SNS: resource-request-event	Asynchronous (Publish)	หลัง Lambda insert DB สำเร็จ	Outbound	event: resource_request.created_success
SNS: to-prioritize-service	Asynchronous (Publish)	หลัง Lambda insert DB สำเร็จ (ทำพร้อมกับ event)	Outbound	ส่งให้ Rescue Prioritize Service
Rescue Prioritize Service	Asynchronous (Inbound via SQS)	Prioritize Service ส่ง priority result กลับ	Inbound	อัปเดต priority ใน DB
Stock Service	Reference ID เท่านั้น	ไม่มีการเรียกจริง	—	Out-of-scope ตาม Service Boundary
Rescue Team Service	Reference ID เท่านั้น	ไม่มีการเรียกจริง	—	Out-of-scope ตาม Service Boundary

1.1 Synchronous Interactions

Incident Tracking Service

- **เรียกเมื่อ:** ทุกครั้งที่ API ได้รับ POST /v1/resource (async path)
- **Endpoint:** GET \${INCIDENT_SERVICE_URL}/incidents/{incident_id}
- **Headers:** X-IncidentTNX-Id (traceld), api-key: 123 (hardcoded — ควร migrate ไป env var ก่อน production)
- **Timeout:** 5 วินาที
- **Fail-safe:** ถ้า timeout หรือ 404 จะเซต verified: false แล้วส่งต่อ SNS ได้ปกติ — ไม่ block การรับเรื่อง

Redis (Valkey) — Idempotency Check

- **เรียกเมื่อ:** request มี header Idempotency-Key
- **Logic:** SET key "processing" NX EX 30 — ถ้าคีย์ซ้ำ return 409 ทันที
- **ใช้กับ endpoint:** POST /v1/resource, POST /:id/assign

PostgreSQL — Synchronous Read

- **ใช้สำหรับ:** GET /list-request, GET /:request_id, PATCH /:id/close, PATCH /:id/unassign, POST /:id/assign
- **ที่ตั้ง:** self-hosted บน Hetzner VPS เดียวกับ API

1.2 Asynchronous Interactions

Outbound: สร้างคำร้องขอใหม่ (API → SNS → SQS → Lambda)

- API รับ POST แล้ว publish event ไปที่ SNS topic resource-request ทันที
- SNS fan-out ไปยัง SQS: resource-create-queue
- Lambda resource-request-inserter consume จาก SQS และทำ 3 อย่างพร้อมกัน:
 - INSERT ลง PostgreSQL (with onConflictDoNothing เป็น idempotency อีกชั้น)
 - Publish resource_request.created_success ไปยัง SNS: resource-request-event
 - Publish payload ไปยัง SNS: to-prioritize-service

Inbound: รับ Priority Result (Rescue Prioritize Service → SQS → Lambda)

- Rescue Prioritize Service publish priority result ไปยัง SQS: prioritization-event-queue
- Lambda resource-request-priority-updater consume แล้ว UPDATE priority ใน DB
- Publish resource_request.priority_updated ไปยัง SNS: resource-request-event

Dependency Mapping

2. Dependency Mapping Table

Dependency	ประเภท (Type)	ทิศทาง (Direction)	ระดับความสำคัญ	Fallback / ผลกระทบหาก เกิดระบบล่ม (Fallback Behavior)
PostgreSQL	Infrastructure (Internal)	Sync / Async	CRITICAL	หาก DB ล่ม: Async Flow จะยังคงสามารถรับ POST Request ได้ โดยข้อมูลจะ ถูกเก็บไว้ที่ SQS Queue รอ ประมวลผลเมื่อ DB กลับมา ทำงาน
Redis (Valkey)	Infrastructure (Internal)	Sync Out	CRITICAL	หาก Redis ล่ม: ระบบจะ ยอมให้ผ่าน (Skip key check/validation) เพื่อให้ POST API ยังทำงานได้ต่อไป แต่ชั่วคราวจะไม่มีระบบ cache
SNS: resource-request	AWS Messaging	Async Out	CRITICAL	หาก SNS ล่ม: ระบบจะทำ การ Return REJECTED status กลับไปยัง Client ทันทีเนื่องจากเป็นท่อส่งหลัก
SQS: resource-create-queue	AWS Messaging	Async	CRITICAL	เป็น AWS Managed Service มีระบบ Auto-retry ในตัว หากมีข้อ ผิดพลาดข้อมูลจะยังคงอยู่ใน คิวตามระยะเวลา Retention Period
Lambda: async_insert_request	AWS Serverless	Async	CRITICAL	หาก Lambda ล่ม: SQS จะยังคงเก็บ Message ไว้ (Message Lock/Visibility Timeout) และจะส่งมา ประมวลผลซ้ำเมื่อ Lambda กลับมาปกติ

SNS: resource-request-event	AWS Messaging	Async Out	HIGH	ระบบจะทำเพียงการ Log Error และจะไม่ส่ง Throw Exception ขึ้นไป เพื่อให้กระบวนการหลัก (Main insert flow) ทำงานเสร็จสิ้นสมบูรณ์
SNS: to-prioritize-service	AWS Messaging	Async Out	HIGH	สถานะลำดับความสำคัญ (Priority) ของ Resource จะคงไว้ที่สถานะ UNDECIDED หากบริการตรวจสอบ Priority ไม่สามารถติดต่อได้
SQS: prioritization-event-queue	AWS Messaging	Async In	HIGH	ข้อมูลจัดลำดับจะไม่ถูกอัปเดต (คงสถานะ UNDECIDED) โดยข้อความยังรอค้างอยู่ใน Queue ของ AWS
Lambda: update_priority_from_queue	AWS Serverless	Async In	HIGH	SQS จะเก็บข้อความไว้ และพยายามส่งให้ Lambda ทำการประมวลผลใหม่อีกครั้งเมื่อบริการกลับมาใช้งานได้
Incident Tracking Service	External Service	Sync Out	OPTIONAL	หากระบบล้ม: จะกำหนดค่า verified = false ใน payload และระบบยังสามารถทำการต่อเพื่อส่งไปยัง SNS ได้ตามปกติ